STUDENT LABORATORY MANUAL

Module 6: Introduction to Computer Graphics and Visual Computing

20 Guided Examples • 20 Practice Exercises • 20 Outputs

Course: CS0045 – Computer Graphics and Visual Computing

Companion: Module 6 Student Guide

Toolchain: VS Code + MSYS2 UCRT64 GCC + GLEW + FreeGLUT

Student Name:	**Ebron, Nicole Anne V.**
Section:	**TN35**
Professor:	**Dr. Hadji J. Tejuco**

GitHub Repository: https://github.com/nicoleanneebron-art/Ebron-Nicole-Anne-V.

How to Use This Manual

This manual has three parts:

- Part A — 20 worked Examples (Ex01–Ex20). Each is a complete, compiled, ready-to-run program demonstrating one concept at a time: immediate-mode and VBO 3D shapes, the translate()/scale()/rotateZ()/rotateY() matrix functions applied to GPU-resident data, combined transformations, glBufferSubData color objects, glArrayElement loops, error handling, and a final animated capstone. Type or copy each one into your SOURCE CODE\M6 folder, compile it (Ctrl+Shift+B), and run it (F5) before moving to the next.
- Part B — 20 Practice Exercises (Q01–Q20), split into Easy (Q01–Q07), Medium (Q08–Q14), and Hard (Q15–Q20). Each exercise gives you an Objective, a Task question describing exactly what your program must do, and Hints to point you in the right direction. Read the task and hints, then write the program YOURSELF before checking Part C.
- Part C — the full Answer Key for every exercise in Part B, with complete, compiled, working code.

Setup reminder

This module builds on Module 5's GLEW + FreeGLUT setup. On Windows/MSYS2 UCRT64: pacman -S mingw-w64-ucrt-x86_64-glew. Include <GL/glew.h> BEFORE <GL/glut.h>, and call glewInit() after glutCreateWindow()/glutDisplayFunc() but before glutMainLoop(). All programs in this manual use the same cross-platform header block shown below.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include  <iostream>
using namespace std;
```

Every program in this manual has been compiled and verified with g++ against GLEW and FreeGLUT.

Part A — Guided Examples (Ex01–Ex20)

Example 01 — Immediate Mode Color Cube (Before Conversion)

Concept: A plain glBegin(GL_QUADS) cube with three visible faces, each a flat color — the starting point before we convert this shape to a VBO.

Expected output: A cube showing a green top face, an orange bottom-adjacent face, and a red front face.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;


// Immediate-mode color cube: the starting point before we convert
// this shape to vertex arrays and VBOs in later examples.
void displayCube(){
    glClear(GL_COLOR_BUFFER_BIT);

    glBegin(GL_QUADS);
        // Top face (y = 1.0f) - Green
        glColor3f(0.0f, 1.0f, 0.0f);
        glVertex3f( 1.0f, 1.0f, -1.0f);
        glVertex3f(-1.0f, 1.0f, -1.0f);
        glVertex3f(-1.0f, 1.0f,  1.0f);
        glVertex3f( 1.0f, 1.0f,  1.0f);

        // Bottom face (y = -1.0f) - Orange
        glColor3f(1.0f, 0.5f, 0.0f);
        glVertex3f( 1.0f, -1.0f,  1.0f);
        glVertex3f(-1.0f, -1.0f,  1.0f);
        glVertex3f(-1.0f, -1.0f, -1.0f);
        glVertex3f( 1.0f, -1.0f, -1.0f);

        // Front face (z = 1.0f) - Red
        glColor3f(1.0f, 0.0f, 0.0f);
        glVertex3f( 1.0f,  1.0f, 1.0f);
        glVertex3f(-1.0f,  1.0f, 1.0f);
        glVertex3f(-1.0f, -1.0f, 1.0f);
        glVertex3f( 1.0f, -1.0f, 1.0f);
    glEnd();
```

```
    glFlush();
}
void display(){ displayCube(); }

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 02 — Immediate Mode Pyramid (Before Conversion)

Concept: A plain glBegin(GL_TRIANGLES) pyramid where every VERTEX gets its own color, so OpenGL blends the colors smoothly across each face.

Expected output: A pyramid with two visible triangular faces, each blending between red, green, and blue.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;


// Immediate-mode 4-triangle pyramid: the starting point before we
// convert this shape to vertex arrays and VBOs in later examples.
void displayTriangle(){
    glClear(GL_COLOR_BUFFER_BIT);

    glBegin(GL_TRIANGLES);
        // Front
        glColor3f(1.0f, 0.0f, 0.0f);
        glVertex3f( 0.0f,  1.0f, 0.0f);
```

```
        glColor3f(0.0f, 1.0f, 0.0f);
        glVertex3f(-1.0f, -1.0f, 1.0f);
        glColor3f(0.0f, 0.0f, 1.0f);
        glVertex3f( 1.0f, -1.0f, 1.0f);

        // Right
        glColor3f(1.0f, 0.0f, 0.0f);
        glVertex3f(0.0f, 1.0f, 0.0f);
        glColor3f(0.0f, 0.0f, 1.0f);
        glVertex3f(1.0f, -1.0f, 1.0f);
        glColor3f(0.0f, 1.0f, 0.0f);
        glVertex3f(1.0f, -1.0f, -1.0f);
    glEnd();

    glFlush();
}
void display(){ displayTriangle(); }

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE__
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 03 — The Cube, as a VBO

Concept: Converts two of the cube's faces to a single VBO using the standard 5-step pattern, drawing each face with its own glColor3f() call.

Expected output: The same two cube faces from Example 1, now sourced from GPU-resident vertex data.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
```

```
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))

// The same cube shape, this time stored as a single VBO of position
// data only, drawn one face at a time with GL_QUADS.
GLfloat cubeVertices[] = {
    // Top
     1.0f, 1.0f, -1.0f,
    -1.0f, 1.0f, -1.0f,
    -1.0f, 1.0f,  1.0f,
     1.0f, 1.0f,  1.0f,
    // Front
     1.0f,  1.0f, 1.0f,
    -1.0f,  1.0f, 1.0f,
    -1.0f, -1.0f, 1.0f,
     1.0f, -1.0f, 1.0f,
};

void displayObject(){
    GLuint VBOid;
    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 24, cubeVertices, GL_STATIC_DRAW);

    glEnableClientState(GL_VERTEX_ARRAY);
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(0.0f, 1.0f, 0.0f);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glDrawArrays(GL_QUADS, 0, 4);

    glColor3f(1.0f, 0.0f, 0.0f);
    glVertexPointer(3, GL_FLOAT, 0, BUFFER_OFFSET(12 * sizeof(GLfloat)));
    glDrawArrays(GL_QUADS, 0, 4);

    glDisableClientState(GL_VERTEX_ARRAY);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);

    glFlush();
}
void display(){ displayObject(); }

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);
```

```
#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 04 — The Pyramid, as a VBO

Concept: Converts the pyramid's front and right faces to a single VBO of position data, drawn with GL_TRIANGLES.

Expected output: The pyramid's front and right faces in a single flat color, sourced from a VBO.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;


// The pyramid's front and right faces, stored as a single VBO of
// position data, drawn with GL_TRIANGLES.
GLfloat pyramidVertices[] = {
     0.0f,  1.0f, 0.0f,
    -1.0f, -1.0f, 1.0f,
     1.0f, -1.0f, 1.0f,

     0.0f,  1.0f,  0.0f,
     1.0f, -1.0f,  1.0f,
     1.0f, -1.0f, -1.0f,
};

void displayObject(){
    GLuint VBOid;
    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 18, pyramidVertices,
GL_STATIC_DRAW);
```

```
    glEnableClientState(GL_VERTEX_ARRAY);
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(1.0f, 1.0f, 0.0f);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glDrawArrays(GL_TRIANGLES, 0, 6);

    glDisableClientState(GL_VERTEX_ARRAY);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);

    glFlush();
}
void display(){ displayObject(); }

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE__
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 05 — The translate() Matrix Function, Applied Once

Concept: Builds the lecture's translate() function and calls it once per vertex via glMapBuffer, shifting a VBO triangle by a fixed amount before the first draw.

Expected output: A green triangle, shifted 0.3 units to the right of where its raw array data would normally place it.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
```

```
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))

// Applies the lecture's translate() matrix function to a VBO triangle
// ONE TIME using glMapBuffer, moving it 0.3 units in X before drawing.
GLuint VBOid;
bool ready = false;
GLfloat triVertices[] = {
     0.0f,  0.5f, 0.0f,
    -0.5f, -0.5f, 0.0f,
     0.5f, -0.5f, 0.0f,
};

// Translation matrix template, from the lecture's Matrix Translation slide
void translate(float x, float y, float z, int startindex){
    GLfloat *mapV = (GLfloat*) glMapBuffer(GL_ARRAY_BUFFER, GL_READ_WRITE);

    GLfloat w = 1.0f;
    GLfloat ta[] = {
        1.0f, 0.0f, 0.0f, x,
        0.0f, 1.0f, 0.0f, y,
        0.0f, 0.0f, 1.0f, z,
        0.0f, 0.0f, 0.0f, 1
    };

    GLfloat tx, ty, tz;
    tx = (ta[0]*mapV[startindex]) + (ta[1]*mapV[startindex+1]) +
(ta[2]*mapV[startindex+2]) + (ta[3]*w);
    ty = (ta[4]*mapV[startindex]) + (ta[5]*mapV[startindex+1]) +
(ta[6]*mapV[startindex+2]) + (ta[7]*w);
    tz = (ta[8]*mapV[startindex]) + (ta[9]*mapV[startindex+1]) +
(ta[10]*mapV[startindex+2]) + (ta[11]*w);

    mapV[startindex]  =  tx;
    mapV[startindex+1] = ty;
    mapV[startindex+2] = tz;

    glUnmapBuffer(GL_ARRAY_BUFFER);
}

void triangle(){
    if(!ready){
        glGenBuffers(1, &VBOid);
        glBindBuffer(GL_ARRAY_BUFFER, VBOid);
        glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 9, triVertices,
GL_DYNAMIC_DRAW);

        for(int i = 0; i < 9; i += 3){
```

```
            translate(0.3f, 0.0f, 0.0f, i);
        }
        ready = true;
    } else {
        glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    }

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glDisableClientState(GL_VERTEX_ARRAY);
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(.16f, .72f, .08f);
    triangle();
    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 06 — Continuous Matrix Translation via glutTimerFunc

Concept: The full lecture example: a hexagon that shifts right once per second using translate() + glMapBuffer, stopping and cleaning up its buffer after 10 updates.

Expected output: A green hexagon that visibly steps to the right once per second, then the program exits after the 10th step.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
```

```
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))

// The lecture's full "Converted Program with Matrix Translation" example:
// a hexagon that shifts to the right once per second via translate(),
// stopping and cleaning up its buffer after 10 updates.
void display();
void translate(float x, float y, float z, int startindex);

GLfloat polygonvertices[] = {
    -0.5f, 0.5f, 0.0f,
     0.5f, 0.5f, 0.0f,
     1.0f, 0.0f, 0.0f,
     0.5f,-0.5f, 0.0f,
    -0.5f,-0.5f, 0.0f,
    -1.0f, 0.0f, 0.0f
};

GLfloat *mapV = NULL;
int forUpdate = 0;
int ctr = 0;
GLuint VBOid;

void polygon(){
    if(!forUpdate){
        glGenBuffers(1, &VBOid);
        glBindBuffer(GL_ARRAY_BUFFER, VBOid);
        glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 18, polygonvertices,
GL_DYNAMIC_DRAW);

        glEnableClientState(GL_VERTEX_ARRAY);
        glVertexPointer(3, GL_FLOAT, 0, 0);
        glDrawArrays(GL_POLYGON, 0, 6);
        glDisableClientState(GL_VERTEX_ARRAY);
    } else {
        glEnableClientState(GL_VERTEX_ARRAY);
        glVertexPointer(3, GL_FLOAT, 0, 0);
        glDrawArrays(GL_POLYGON, 0, 6);
        glDisableClientState(GL_VERTEX_ARRAY);
    }
}

void translate(float x, float y, float z, int startindex){
    GLfloat w = 1.0f;
    GLfloat ta[] = {
        1.0f, 0.0f, 0.0f, x,
```

```
        0.0f, 1.0f, 0.0f, y,
        0.0f, 0.0f, 1.0f, z,
        0.0f, 0.0f, 0.0f, 1
    };

    GLfloat tx, ty, tz;
    tx = (ta[0]*mapV[startindex]) + (ta[1]*mapV[startindex+1]) +
(ta[2]*mapV[startindex+2]) + (ta[3]* w);
    ty = (ta[4]*mapV[startindex]) + (ta[5]*mapV[startindex+1]) +
(ta[6]*mapV[startindex+2]) + (ta[7]* w);
    tz = (ta[8]*mapV[startindex]) + (ta[9]*mapV[startindex+1]) +
(ta[10]*mapV[startindex+2]) + (ta[11]* w);

    mapV[startindex] = tx;
    mapV[startindex+1] = ty;
    mapV[startindex+2] = tz;
}

void modifyPolygon(int value){
    if(value == 10){
        glBindBuffer(GL_ARRAY_BUFFER, 0);
        glDeleteBuffers(1, &VBOid);
        exit(0);
    }

    forUpdate = 1;

    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    mapV = (GLfloat *) glMapBuffer(GL_ARRAY_BUFFER, GL_READ_WRITE);

    for(int i = 0; i < 18; i += 3){
        translate(.1f, 0.0f, 0.0f, i);
    }

    glUnmapBuffer(GL_ARRAY_BUFFER);
    glutPostRedisplay();

    glutTimerFunc(1000, modifyPolygon, value + 1);
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glColor4f(.16f, .72f, .08f, 1.0f);
    polygon();
    glFlush();
    if(!forUpdate){
        glutTimerFunc(1000, modifyPolygon, 0);
    }
}

int main(int argc, char** argv){
```

```
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 07 — The scale() Matrix Function

Concept: A scaling matrix applied once via glMapBuffer, using the identical code shape as translate() but with a different 16-value matrix template.

Expected output: A triangle 1.5x larger than its raw array data would normally produce.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))

// Applies a scaling matrix (per the lecture's Scaling slide) to a VBO
// triangle ONE TIME via glMapBuffer, growing it by 1.5x before drawing.
GLuint VBOid;
bool ready = false;
GLfloat triVertices[] = {
     0.0f,  0.3f, 0.0f,
    -0.3f, -0.3f, 0.0f,
     0.3f, -0.3f, 0.0f,
};

// Scaling matrix template, from the lecture's Scaling slide
void scale(float sx, float sy, float sz, int startindex){
    GLfloat *mapV = (GLfloat*) glMapBuffer(GL_ARRAY_BUFFER, GL_READ_WRITE);
```

```
    GLfloat sa[] = {
        sx,   0.0f, 0.0f, 0.0f,
        0.0f, sy,   0.0f, 0.0f,
        0.0f, 0.0f, sz,   0.0f,
        0.0f, 0.0f, 0.0f, 1.0f
    };
    GLfloat w = 1.0f;

    GLfloat sx_, sy_, sz_;
    sx_ = (sa[0]*mapV[startindex]) + (sa[1]*mapV[startindex+1]) +
(sa[2]*mapV[startindex+2]) + (sa[3]*w);
    sy_ = (sa[4]*mapV[startindex]) + (sa[5]*mapV[startindex+1]) +
(sa[6]*mapV[startindex+2]) + (sa[7]*w);
    sz_ = (sa[8]*mapV[startindex]) + (sa[9]*mapV[startindex+1]) +
(sa[10]*mapV[startindex+2]) + (sa[11]*w);

    mapV[startindex]  =  sx_;
    mapV[startindex+1] = sy_;
    mapV[startindex+2] = sz_;

    glUnmapBuffer(GL_ARRAY_BUFFER);
}

void triangle(){
    if(!ready){
        glGenBuffers(1, &VBOid);
        glBindBuffer(GL_ARRAY_BUFFER, VBOid);
        glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 9, triVertices,
GL_DYNAMIC_DRAW);

        for(int i = 0; i < 9; i += 3){
            scale(1.5f, 1.5f, 1.5f, i);
        }
        ready = true;
    } else {
        glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    }

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glDisableClientState(GL_VERTEX_ARRAY);
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(.86f, .32f, .08f);
    triangle();
    glFlush();
}
```

```
int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 08 — The rotateZ() Matrix Function, Animated

Concept: A Rz(theta) rotation matrix applied once per second via glutTimerFunc, spinning a triangle continuously around the Z axis.

Expected output: A blue triangle that visibly rotates 15 degrees every second.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))

// Applies the lecture's Rz(theta) rotation matrix to a VBO triangle,
// once per second via glutTimerFunc, spinning it around the Z axis.
GLuint VBOid;
bool ready = false;
GLfloat triVertices[] = {
     0.0f,  0.5f, 0.0f,
    -0.5f, -0.5f, 0.0f,
     0.5f, -0.5f, 0.0f,
};

// Rz(theta) rotation matrix template, from the lecture's Rotation slide
```

```
void rotateZ(float degrees, int startindex){
    GLfloat *mapV = (GLfloat*) glMapBuffer(GL_ARRAY_BUFFER, GL_READ_WRITE);

    float rad = degrees * 3.14159265f / 180.0f;
    GLfloat c = cosf(rad), s = sinf(rad);
    GLfloat ra[] = {
        c,   -s,   0.0f, 0.0f,
        s,    c,   0.0f, 0.0f,
        0.0f, 0.0f, 1.0f, 0.0f,
        0.0f, 0.0f, 0.0f, 1.0f
    };
    GLfloat w = 1.0f;

    GLfloat rx, ry, rz;
    rx = (ra[0]*mapV[startindex]) + (ra[1]*mapV[startindex+1]) +
(ra[2]*mapV[startindex+2]) + (ra[3]*w);
    ry = (ra[4]*mapV[startindex]) + (ra[5]*mapV[startindex+1]) +
(ra[6]*mapV[startindex+2]) + (ra[7]*w);
    rz = (ra[8]*mapV[startindex]) + (ra[9]*mapV[startindex+1]) +
(ra[10]*mapV[startindex+2]) + (ra[11]*w);

    mapV[startindex]  =  rx;
    mapV[startindex+1] = ry;
    mapV[startindex+2] = rz;

    glUnmapBuffer(GL_ARRAY_BUFFER);
}

void triangle(){
    if(!ready){
        glGenBuffers(1, &VBOid);
        glBindBuffer(GL_ARRAY_BUFFER, VBOid);
        glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat) * 9, triVertices,
GL_DYNAMIC_DRAW);
        ready = true;
    } else {
        glBindBuffer(GL_ARRAY_BUFFER,  VBOid);
    }

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glDisableClientState(GL_VERTEX_ARRAY);
}

void spin(int value){
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    for(int i = 0; i < 9; i += 3){
        rotateZ(15.0f, i);
    }
    glutPostRedisplay();
    glutTimerFunc(1000, spin, 0);
```

```
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(.08f, .32f, .86f);
    triangle();
    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutTimerFunc(1000, spin, 0);
        glutMainLoop();
    }
#else
        glutTimerFunc(1000, spin, 0);
    glutMainLoop();
#endif
}
```

Example 09 — Matrix-Vector Product, By Hand

Concept: A pure demonstration of the 4x4-matrix-times-4x1-vector multiplication from the lecture's Matrix Vector Product slide, with the result printed to the console AND drawn as two points.

Expected output: Two points on screen (before and after translation), plus console output showing the numeric result.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;
```

```
// A pure "by hand" demonstration of the matrix-vector product from
// the lecture's Matrix Vector Product slide: multiplying a 4x4 matrix
// by a (x,y,z,w) vector, printed to the console AND drawn as two points.
struct Vec4 { float x, y, z, w; };

Vec4 matrixVectorProduct(const float m[16], Vec4 v){
    Vec4 r;
    r.x = m[0]*v.x + m[1]*v.y + m[2]*v.z  + m[3]*v.w;
    r.y = m[4]*v.x + m[5]*v.y + m[6]*v.z  + m[7]*v.w;
    r.z = m[8]*v.x + m[9]*v.y + m[10]*v.z + m[11]*v.w;
    r.w = m[12]*v.x + m[13]*v.y + m[14]*v.z + m[15]*v.w;
    return r;
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);

    // A translation matrix that moves a point by (0.4, 0.2, 0.0)
    float m[16] = {
        1.0f, 0.0f, 0.0f, 0.4f,
        0.0f, 1.0f, 0.0f, 0.2f,
        0.0f, 0.0f, 1.0f, 0.0f,
        0.0f, 0.0f, 0.0f, 1.0f
    };
    Vec4 before = { -0.4f, -0.2f, 0.0f, 1.0f };
    Vec4 after  = matrixVectorProduct(m, before);

    cout << "Before: (" << before.x << ", " << before.y << ", " << before.z << ")" <<
endl;
    cout << "After:  (" << after.x  << ", " << after.y  << ", " << after.z  << ")" <<
endl;

    glPointSize(10.0f);
    glBegin(GL_POINTS);
        glColor3f(1.0f, 1.0f, 1.0f);
        glVertex3f(before.x, before.y, before.z);
        glColor3f(1.0f, 0.0f, 0.0f);
        glVertex3f(after.x, after.y, after.z);
    glEnd();

    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE__
    GLenum err = glewInit();
    if(err == GLEW_OK){
```

```
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 10 — Homogeneous Coordinates: w=1 vs. w=0

Concept: Demonstrates that a translation matrix moves a POSITION (w=1) but has NO effect on a DIRECTION (w=0), per the lecture's Homogeneous Coordinates rule.

Expected output: Two green/yellow points showing a position before and after translation; console output confirming the direction vector was unaffected.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;


// Demonstrates the lecture's Homogeneous Coordinates rule:
// w == 1 means the vector is a POSITION; w == 0 means a DIRECTION.
// A translation matrix moves a position but leaves a direction unchanged.
struct Vec4 { float x, y, z, w; };

Vec4 applyTranslation(Vec4 v, float tx, float ty, float tz){
    // Only affects x,y,z when w == 1 (position); has no effect when w == 0 (direction)
    Vec4 r;
    r.x = v.x + tx * v.w;
    r.y = v.y + ty * v.w;
    r.z = v.z + tz * v.w;
    r.w = v.w;
    return r;
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);

    Vec4 position  = { -0.3f, -0.3f, 0.0f, 1.0f }; // a point in space
    Vec4 direction = {  0.3f,  0.3f, 0.0f, 0.0f }; // a direction vector
```

```
    Vec4 movedPosition  = applyTranslation(position,  0.3f, 0.0f, 0.0f);
    Vec4 movedDirection = applyTranslation(direction, 0.3f, 0.0f, 0.0f);

    cout << "Position moved from (" << position.x << "," << position.y
         << ") to (" << movedPosition.x << "," << movedPosition.y << ")" << endl;
    cout << "Direction stayed at (" << movedDirection.x << "," << movedDirection.y
         << ") -- unaffected because w == 0" << endl;

    glPointSize(12.0f);
    glBegin(GL_POINTS);
        glColor3f(0.0f, 1.0f, 0.0f);
        glVertex3f(position.x, position.y, 0.0f);
        glColor3f(1.0f, 1.0f, 0.0f);
        glVertex3f(movedPosition.x, movedPosition.y, 0.0f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 11 — glBufferSubData(): Position + Color, One Buffer

Concept: Reserves space for both position and color with one glBufferData(NULL) call, then uploads each with glBufferSubData(), reading them back with glArrayElement().

Expected output: A triangle with red, green, and blue corners, sourced from a single combined VBO.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
```

```
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))

// Uses glBufferData to reserve space for BOTH position and color data,
// then glBufferSubData to upload each block separately -- the pattern
// from the lecture's "glArrayElement(), glColorPointer() and
// glBufferSubData()" slide.
GLfloat triVertices[] = {
     0.0f,  0.5f, 0.0f,
    -0.5f, -0.5f, 0.0f,
     0.5f, -0.5f, 0.0f,
};
GLfloat triColors[] = {
    1.0f, 0.0f, 0.0f,
    0.0f, 1.0f, 0.0f,
    0.0f, 0.0f, 1.0f,
};

void displayObject(){
    GLuint VBOid;
    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);

    // Reserve space for 9 position floats + 9 color floats
    glBufferData(GL_ARRAY_BUFFER, (9 + 9) * sizeof(GLfloat), NULL, GL_STATIC_DRAW);
    glBufferSubData(GL_ARRAY_BUFFER, 0, 9 * sizeof(GLfloat), triVertices);
    glBufferSubData(GL_ARRAY_BUFFER, 9 * sizeof(GLfloat), 9 * sizeof(GLfloat),
triColors);

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);

    glClear(GL_COLOR_BUFFER_BIT);

    glVertexPointer(3, GL_FLOAT, 0, 0);
    glColorPointer(3, GL_FLOAT, 0, BUFFER_OFFSET(9 * sizeof(GLfloat)));

    glBegin(GL_TRIANGLES);
        glArrayElement(0);
        glArrayElement(1);
        glArrayElement(2);
    glEnd();

    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);
```

```
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);

    glFlush();
}
void display(){ displayObject(); }

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE__
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 12 — glBufferSubData() With glDrawArrays

Concept: The same position+color glBufferSubData pattern, this time read out with glColorPointer + glDrawArrays(GL_QUADS, …) instead of glArrayElement.

Expected output: A quad blending from yellow at the top to blue at the bottom.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))

// The lecture's colored-square follow-up: position + color for a quad,
// both uploaded with glBufferSubData, drawn with glDrawArrays(GL_QUADS,...).
GLfloat quadVertices[] = {
    -0.5f,  0.5f, 0.0f,
     0.5f,  0.5f, 0.0f,
```

```
     0.5f, -0.5f, 0.0f,
    -0.5f, -0.5f, 0.0f,
};
GLfloat quadColors[] = {
    1.0f, 1.0f, 0.0f,
    1.0f, 1.0f, 0.0f,
    0.0f, 0.0f, 1.0f,
    0.0f, 0.0f, 1.0f,
};

void displayObject(){
    GLuint VBOid;
    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);

    glBufferData(GL_ARRAY_BUFFER, (12 + 12) * sizeof(GLfloat), NULL, GL_STATIC_DRAW);
    glBufferSubData(GL_ARRAY_BUFFER, 0, 12 * sizeof(GLfloat), quadVertices);
    glBufferSubData(GL_ARRAY_BUFFER, 12 * sizeof(GLfloat), 12 * sizeof(GLfloat),
quadColors);

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);

    glClear(GL_COLOR_BUFFER_BIT);

    glVertexPointer(3, GL_FLOAT, 0, 0);
    glColorPointer(3, GL_FLOAT, 0, BUFFER_OFFSET(12 * sizeof(GLfloat)));
    glDrawArrays(GL_QUADS, 0, 4);

    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);

    glFlush();
}
void display(){ displayObject(); }

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
```

```
    glutMainLoop();
#endif
}
```

Example 13 — Color Cube Faces via Position + Color VBO

Concept: Two cube faces where EVERY vertex has its own color, combined into one VBO via glBufferSubData.

Expected output: A green top face and a red front face, each face uniformly colored via its per-vertex color data.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))

// Two faces of the lecture's color cube, this time using a position VBO
// PLUS a color VBO (via glBufferSubData) so every vertex has its own color.
GLfloat cubeVertices[] = {
    // Top face - 4 verts
     1.0f, 1.0f, -1.0f,
    -1.0f, 1.0f, -1.0f,
    -1.0f, 1.0f,  1.0f,
     1.0f, 1.0f,  1.0f,
    // Front face - 4 verts
     1.0f,  1.0f, 1.0f,
    -1.0f,  1.0f, 1.0f,
    -1.0f, -1.0f, 1.0f,
     1.0f, -1.0f, 1.0f,
};
GLfloat cubeColors[] = {
    0.0f, 1.0f, 0.0f,  0.0f, 1.0f, 0.0f,  0.0f, 1.0f, 0.0f,  0.0f, 1.0f, 0.0f,
    1.0f, 0.0f, 0.0f,  1.0f, 0.0f, 0.0f,  1.0f, 0.0f, 0.0f,  1.0f, 0.0f, 0.0f,
};

void displayObject(){
    GLuint VBOid;
    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
```

```
    glBufferData(GL_ARRAY_BUFFER, (24 + 24) * sizeof(GLfloat), NULL, GL_STATIC_DRAW);
    glBufferSubData(GL_ARRAY_BUFFER, 0, 24 * sizeof(GLfloat), cubeVertices);
    glBufferSubData(GL_ARRAY_BUFFER, 24 * sizeof(GLfloat), 24 * sizeof(GLfloat),
cubeColors);

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);

    glClear(GL_COLOR_BUFFER_BIT);

    glVertexPointer(3, GL_FLOAT, 0, 0);
    glColorPointer(3, GL_FLOAT, 0, BUFFER_OFFSET(24 * sizeof(GLfloat)));

    glDrawArrays(GL_QUADS, 0, 8);

    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);

    glFlush();
}
void display(){ displayObject(); }

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE__
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 14 — Combining Transformations: One Multiply, Two Effects

Concept: Multiplies a translation matrix and a scale matrix into ONE combined 4x4 matrix using a general multiply4x4() helper, then applies it to a single point.

Expected output: Two points (before and after) plus console output showing the combined matrix's numeric result.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;


// Reproduces the lecture's "Combining Transformation" slide: multiplying
// a translation matrix by a scale matrix into ONE combined matrix, then
// applying that single combined matrix to a vertex.
void multiply4x4(const float a[16], const float b[16], float out[16]){
    for(int row = 0; row < 4; row++){
        for(int col = 0; col < 4; col++){
            out[row*4 + col] = 0.0f;
            for(int k = 0; k < 4; k++){
                out[row*4 + col] += a[row*4 + k] * b[k*4 + col];
            }
        }
    }
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);

    // Translate by (0.2, 0.1, 0.0)
    float T[16] = {
        1.0f, 0.0f, 0.0f, 0.2f,
        0.0f, 1.0f, 0.0f, 0.1f,
        0.0f, 0.0f, 1.0f, 0.0f,
        0.0f, 0.0f, 0.0f, 1.0f
    };
    // Scale by 1.5 in all directions
    float S[16] = {
        1.5f, 0.0f, 0.0f, 0.0f,
        0.0f, 1.5f, 0.0f, 0.0f,
        0.0f, 0.0f, 1.5f, 0.0f,
        0.0f, 0.0f, 0.0f, 1.0f
    };
    float combined[16];
    multiply4x4(T, S, combined);

    float vx = -0.2f, vy = -0.1f, vz = 0.0f, vw = 1.0f;
    float rx = combined[0]*vx + combined[1]*vy + combined[2]*vz  + combined[3]*vw;
    float ry = combined[4]*vx + combined[5]*vy + combined[6]*vz  + combined[7]*vw;
    float rz = combined[8]*vx + combined[9]*vy + combined[10]*vz + combined[11]*vw;
```

```
    cout << "Result: (" << rx << ", " << ry << ", " << rz << ")" << endl;

    glPointSize(12.0f);
    glBegin(GL_POINTS);
        glColor3f(1.0f, 1.0f, 1.0f);
        glVertex3f(vx, vy, vz);
        glColor3f(1.0f, 0.5f, 0.0f);
        glVertex3f(rx, ry, rz);
    glEnd();

    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 15 — glArrayElement() in a Loop

Concept: Replaces hand-typed glArrayElement() calls with a for loop, per the lecture's "you can just create a loop for all the vertices" comment.

Expected output: A six-color hexagon, each corner a different color, drawn via a loop of glArrayElement() calls.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;
```

```
#define BUFFER_OFFSET(i) ((void*)(i))

// Uses glArrayElement() inside glBegin/glEnd with a FOR LOOP instead of
// hand-typing each call, per the lecture's comment: "you can just create
// a loop for all the vertices."
GLfloat hexVertices[] = {
    -0.5f, 0.25f, 0.0f,
     0.0f, 0.5f,  0.0f,
     0.5f, 0.25f, 0.0f,
     0.5f,-0.25f, 0.0f,
     0.0f,-0.5f,  0.0f,
    -0.5f,-0.25f, 0.0f,
};
GLfloat hexColors[] = {
    1.0f,0.0f,0.0f,  1.0f,0.5f,0.0f,  1.0f,1.0f,0.0f,
    0.0f,1.0f,0.0f,  0.0f,0.0f,1.0f,  0.6f,0.0f,1.0f,
};

void displayObject(){
    GLuint VBOid;
    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);

    glBufferData(GL_ARRAY_BUFFER, (18 + 18) * sizeof(GLfloat), NULL, GL_STATIC_DRAW);
    glBufferSubData(GL_ARRAY_BUFFER, 0, 18 * sizeof(GLfloat), hexVertices);
    glBufferSubData(GL_ARRAY_BUFFER, 18 * sizeof(GLfloat), 18 * sizeof(GLfloat),
hexColors);

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);

    glClear(GL_COLOR_BUFFER_BIT);

    glVertexPointer(3, GL_FLOAT, 0, 0);
    glColorPointer(3, GL_FLOAT, 0, BUFFER_OFFSET(18 * sizeof(GLfloat)));

    glBegin(GL_POLYGON);
        for(int i = 0; i < 6; i++){
            glArrayElement(i);
        }
    glEnd();

    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);

    glFlush();
}
```

```
void display(){ displayObject(); }

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 16 — Registering an Error-Handling Callback

Concept: Registers glDebugMessageCallback per the lecture's Handling Errors slide, so any future OpenGL error prints a diagnostic message.

Expected output: A plain white triangle; the console prints the OpenGL version and vendor strings, with no error messages (since nothing is wrong).

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;


// The lecture's error-handling pattern: register a GL_DEBUG_OUTPUT
// callback so any OpenGL error prints a message instead of failing silently.
void GLAPIENTRY MessageCallback(GLenum source,
                                GLenum type,
                                GLuint id,
                                GLenum severity,
                                GLsizei length,
                                const GLchar* message,
                                const void* userParam){
    fprintf(stderr, "GL CALLBACK: %s type = 0x%x, severity = 0x%x, message = %s\n",
```

```
            (type == GL_DEBUG_TYPE_ERROR ? "** GL ERROR **" : ""),
            type, severity, message);
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 1.0f);
    glBegin(GL_TRIANGLES);
        glVertex2f(-0.5f, -0.5f);
        glVertex2f( 0.5f, -0.5f);
        glVertex2f( 0.0f,  0.5f);
    glEnd();
    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glEnable(GL_DEBUG_OUTPUT);
        glDebugMessageCallback(MessageCallback, 0);
        printf("OpenGL version supported by this platform (%s): \n",
glGetString(GL_VERSION));
        printf("OpenGL vendor (%s): \n", glGetString(GL_VENDOR));
        glutMainLoop();
    }
#else
        glEnable(GL_DEBUG_OUTPUT);
        glDebugMessageCallback(MessageCallback, 0);
        printf("OpenGL version supported by this platform (%s): \n",
glGetString(GL_VERSION));
        printf("OpenGL vendor (%s): \n", glGetString(GL_VENDOR));
    glutMainLoop();
#endif
}
```

Example 17 — Left-Hand vs. Right-Hand: Winding Order and Culling

Concept: Two triangles with opposite winding order, with GL_CULL_FACE enabled, illustrating OpenGL's right-hand coordinate convention.

Expected output: A green counter-clockwise triangle on the left (front-facing, visible) and NO triangle on the right (the clockwise one is culled).

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;


// Illustrates the lecture's Left-Hand vs Right-Hand slide using winding
// order: the same three points listed clockwise vs counter-clockwise
// face the camera differently once back-face culling is enabled.
void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glEnable(GL_CULL_FACE);
    glCullFace(GL_BACK);

    // Left triangle: counter-clockwise winding (front-facing, right-hand rule)
    glColor3f(0.0f, 1.0f, 0.0f);
    glBegin(GL_TRIANGLES);
        glVertex2f(-0.8f, -0.3f);
        glVertex2f(-0.4f, -0.3f);
        glVertex2f(-0.6f,  0.3f);
    glEnd();

    // Right triangle: clockwise winding (back-facing, gets culled -- won't draw)
    glColor3f(1.0f, 0.0f, 0.0f);
    glBegin(GL_TRIANGLES);
        glVertex2f(0.4f, -0.3f);
        glVertex2f(0.8f, -0.3f);
        glVertex2f(0.6f,  0.3f);
    glEnd();

    glDisable(GL_CULL_FACE);
    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
```

```
#else
    glutMainLoop();
#endif
}
```

Example 18 — Full Three-Face Color Cube

Concept: All three visible cube faces (top, front, right) combined into ONE VBO via glBufferSubData, each face a different flat color.

Expected output: A complete 3-visible-face cube: green top, red front, magenta right side.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))

// The full three-visible-face color cube (top, front, right), using
// ONE VBO with position data followed by color data via glBufferSubData.
GLfloat cubeVertices[] = {
    // Top - Green
     1.0f, 1.0f, -1.0f,  -1.0f, 1.0f, -1.0f,  -1.0f, 1.0f, 1.0f,   1.0f, 1.0f, 1.0f,
    // Front - Red
     1.0f, 1.0f,  1.0f,   -1.0f, 1.0f,  1.0f,  -1.0f,-1.0f, 1.0f,   1.0f,-1.0f, 1.0f,
    // Right - Magenta
     1.0f, 1.0f, -1.0f,    1.0f, 1.0f,  1.0f,   1.0f,-1.0f, 1.0f,   1.0f,-1.0f,-1.0f,
};
GLfloat cubeColors[] = {
    0.0f,1.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,1.0f,0.0f,
    1.0f,0.0f,0.0f, 1.0f,0.0f,0.0f, 1.0f,0.0f,0.0f, 1.0f,0.0f,0.0f,
    1.0f,0.0f,1.0f, 1.0f,0.0f,1.0f, 1.0f,0.0f,1.0f, 1.0f,0.0f,1.0f,
};

void displayObject(){
    GLuint VBOid;
    glGenBuffers(1, &VBOid);
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);

    glBufferData(GL_ARRAY_BUFFER, (36 + 36) * sizeof(GLfloat), NULL, GL_STATIC_DRAW);
    glBufferSubData(GL_ARRAY_BUFFER, 0, 36 * sizeof(GLfloat), cubeVertices);
```

```
    glBufferSubData(GL_ARRAY_BUFFER, 36 * sizeof(GLfloat), 36 * sizeof(GLfloat),
cubeColors);

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);

    glClear(GL_COLOR_BUFFER_BIT);

    glVertexPointer(3, GL_FLOAT, 0, 0);
    glColorPointer(3, GL_FLOAT, 0, BUFFER_OFFSET(36 * sizeof(GLfloat)));
    glDrawArrays(GL_QUADS, 0, 12);

    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glDeleteBuffers(1, &VBOid);

    glFlush();
}
void display(){ displayObject(); }

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 19 — rotateY(): Continuous Rotation Around the Y Axis

Concept: A colored triangle (position + color VBO) spun continuously around the Y axis using an Ry(theta) matrix applied once per second.

Expected output: A red/green/blue triangle that visibly rotates around a vertical axis, appearing to narrow and widen as it turns.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
```

```
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))

// A colored triangle spun continuously around the Y axis using the
// Ry(theta) rotation matrix from the lecture's Rotation slide, applied
// through glMapBuffer once per second.
GLuint VBOid;
bool ready = false;
GLfloat triVertices[] = {
     0.0f,  0.5f, 0.0f,
    -0.5f, -0.5f, 0.0f,
     0.5f, -0.5f, 0.0f,
};
GLfloat triColors[] = {
    1.0f, 0.0f, 0.0f,
    0.0f, 1.0f, 0.0f,
    0.0f, 0.0f, 1.0f,
};

// Ry(theta) rotation matrix template, from the lecture's Rotation slide
void rotateY(float degrees, int startindex){
    GLfloat *mapV = (GLfloat*) glMapBuffer(GL_ARRAY_BUFFER, GL_READ_WRITE);

    float rad = degrees * 3.14159265f / 180.0f;
    GLfloat c = cosf(rad), s = sinf(rad);
    GLfloat ra[] = {
        c,    0.0f, s,    0.0f,
        0.0f, 1.0f, 0.0f, 0.0f,
       -s,    0.0f, c,    0.0f,
        0.0f, 0.0f, 0.0f, 1.0f
    };
    GLfloat w = 1.0f;

    GLfloat rx, ry, rz;
    rx = (ra[0]*mapV[startindex]) + (ra[1]*mapV[startindex+1]) +
(ra[2]*mapV[startindex+2]) + (ra[3]*w);
    ry = (ra[4]*mapV[startindex]) + (ra[5]*mapV[startindex+1]) +
(ra[6]*mapV[startindex+2]) + (ra[7]*w);
    rz = (ra[8]*mapV[startindex]) + (ra[9]*mapV[startindex+1]) +
(ra[10]*mapV[startindex+2]) + (ra[11]*w);

    mapV[startindex]   = rx;
    mapV[startindex+1] = ry;
```

```
    mapV[startindex+2] = rz;

    glUnmapBuffer(GL_ARRAY_BUFFER);
}

void triangle(){
    if(!ready){
        glGenBuffers(1, &VBOid);
        glBindBuffer(GL_ARRAY_BUFFER, VBOid);
        glBufferData(GL_ARRAY_BUFFER, (9 + 9) * sizeof(GLfloat), NULL, GL_DYNAMIC_DRAW);
        glBufferSubData(GL_ARRAY_BUFFER, 0, 9 * sizeof(GLfloat), triVertices);
        glBufferSubData(GL_ARRAY_BUFFER, 9 * sizeof(GLfloat), 9 * sizeof(GLfloat),
triColors);
        ready = true;
    } else {
        glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    }

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glColorPointer(3, GL_FLOAT, 0, BUFFER_OFFSET(9 * sizeof(GLfloat)));
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);
}

void spin(int value){
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    for(int i = 0; i < 9; i += 3){
        rotateY(20.0f, i);
    }
    glutPostRedisplay();
    glutTimerFunc(1000, spin, 0);
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    triangle();
    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutTimerFunc(1000, spin, 0);
```

```
        glutMainLoop();
    }
#else
        glutTimerFunc(1000, spin, 0);
    glutMainLoop();
#endif
}
```

Example 20 — Capstone: Animated Color Cube With Cleanup

Concept: A two-face color cube (position + color combined via glBufferSubData) that drifts sideways every second using translate(), with ESC-key cleanup releasing its buffer.

Expected output: A green/red two-face cube that drifts to the right once per second; pressing ESC releases the buffer and exits.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))

// Capstone: a two-face color cube (position VBO + color VBO combined via
// glBufferSubData) that drifts sideways once per second using the
// lecture's translate() matrix helper, with ESC-key cleanup.
GLuint VBOid;
bool ready = false;

GLfloat cubeVertices[] = {
    // Top - Green
     1.0f, 1.0f, -1.0f,  -1.0f, 1.0f, -1.0f,  -1.0f, 1.0f, 1.0f,    1.0f, 1.0f, 1.0f,
    // Front - Red
     1.0f, 1.0f,  1.0f,  -1.0f, 1.0f,  1.0f,  -1.0f,-1.0f, 1.0f,    1.0f,-1.0f, 1.0f,
};
GLfloat cubeColors[] = {
    0.0f,1.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,1.0f,0.0f,
    1.0f,0.0f,0.0f, 1.0f,0.0f,0.0f, 1.0f,0.0f,0.0f, 1.0f,0.0f,0.0f,
};

// Translation matrix template, from the lecture's Matrix Translation slide
void translate(float x, float y, float z, int startindex){
```

```
    GLfloat *mapV = (GLfloat*) glMapBuffer(GL_ARRAY_BUFFER, GL_READ_WRITE);

    GLfloat w = 1.0f;
    GLfloat ta[] = {
        1.0f, 0.0f, 0.0f, x,
        0.0f, 1.0f, 0.0f, y,
        0.0f, 0.0f, 1.0f, z,
        0.0f, 0.0f, 0.0f, 1
    };

    GLfloat tx, ty, tz;
    tx = (ta[0]*mapV[startindex]) + (ta[1]*mapV[startindex+1]) +
(ta[2]*mapV[startindex+2]) + (ta[3]*w);
    ty = (ta[4]*mapV[startindex]) + (ta[5]*mapV[startindex+1]) +
(ta[6]*mapV[startindex+2]) + (ta[7]*w);
    tz = (ta[8]*mapV[startindex]) + (ta[9]*mapV[startindex+1]) +
(ta[10]*mapV[startindex+2]) + (ta[11]*w);

    mapV[startindex]  =  tx;
    mapV[startindex+1] = ty;
    mapV[startindex+2] = tz;

    glUnmapBuffer(GL_ARRAY_BUFFER);
}

void cube(){
    if(!ready){
        glGenBuffers(1, &VBOid);
        glBindBuffer(GL_ARRAY_BUFFER, VBOid);
        glBufferData(GL_ARRAY_BUFFER, (24 + 24) * sizeof(GLfloat), NULL,
GL_DYNAMIC_DRAW);
        glBufferSubData(GL_ARRAY_BUFFER, 0, 24 * sizeof(GLfloat), cubeVertices);
        glBufferSubData(GL_ARRAY_BUFFER, 24 * sizeof(GLfloat), 24 * sizeof(GLfloat),
cubeColors);
        ready = true;
    } else {
        glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    }

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, 0);
    glColorPointer(3, GL_FLOAT, 0, BUFFER_OFFSET(24 * sizeof(GLfloat)));
    glDrawArrays(GL_QUADS, 0, 8);
    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);
}

void drift(int value){
    glBindBuffer(GL_ARRAY_BUFFER, VBOid);
    for(int i = 0; i < 24; i += 3){
        translate(0.02f, 0.0f, 0.0f, i);
```

```
    }
    glutPostRedisplay();
    glutTimerFunc(1000, drift, 0);
}

void monitorKeyboard(unsigned char key, int x, int y){
    if(key == 27){
        glBindBuffer(GL_ARRAY_BUFFER, 0);
        glDeleteBuffers(1, &VBOid);
        exit(0);
    }
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    cube();
    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("simple");
    glutDisplayFunc(display);
    glutKeyboardFunc(monitorKeyboard);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutTimerFunc(1000, drift, 0);
        glutMainLoop();
    }
#else
        glutTimerFunc(1000, drift, 0);
    glutMainLoop();
#endif
}
```

Part B — Practice Exercises (Q01–Q20)

Attempt each exercise on your own first. Compile and run your own solution, compare its output to the task description, and only THEN check Part C. Difficulty is marked Easy, Medium, or Hard. Each exercise's "Task" line is the exact question you must answer with your program.

Easy Exercises

Exercise Q01 — Immediate-Mode 4-Sided Pyramid [Easy]

Objective: Practice drawing a full immediate-mode pyramid with all 4 side faces, each a different flat color.

Task (Question): Draw a pyramid using glBegin(GL_TRIANGLES) with FOUR side faces (front, right, back, left), giving each face its OWN flat color via one glColor3f() call per face.

Hints:

- Each face needs 3 glVertex3f() calls, using the same apex point (0,1,0) plus two of the four base corners.
- Call glColor3f() ONCE per face, right before that face's 3 vertices.

Exercise Q02 — VBO Conversion of One Cube Face [Easy]

Objective: Practice converting a single immediate-mode cube face to a VBO using the standard 5-step pattern.

Task (Question): Convert the cube's TOP face (a GL_QUADS shape) to a VBO using all 5 steps, keeping a single flat glColor3f() color.

Hints:

- You need only ONE VBO here — 4 vertices, position data only.
- Remember glVertexPointer's final argument becomes an offset (0), not a real pointer, once the buffer is bound.

Exercise Q03 — Scalar Product Matrix, By Hand [Easy]

Objective: Practice the scalar-product matrix operation from the lecture's Scalar Product slide.

Task (Question): Multiply a 2x2 scalar matrix (2.0 on the diagonal) against a single 2D point BY HAND in code, print the before/after values to the console, and draw both points.

Hints:

- A 2x2 scalar matrix has the form [[s,0],[0,s]] — multiplying it by (x,y) just gives (s*x, s*y).

- Use cout to print both the input and output coordinates so you can verify the math yourself.

Exercise Q04 — A Simple translateX() Helper [Easy]

Objective: Practice writing your OWN small matrix-style function that shifts a VBO vertex along one axis.

Task (Question): Write a translateX(float x, int startindex) function that adds x to a mapped vertex's X coordinate ONLY (no full matrix multiply needed), and apply it to every vertex of a VBO triangle via glMapBuffer.

Hints:

- You do not need the full 4x4 matrix template for this exercise — just mapV[startindex] += x is enough.
- Remember to call glMapBuffer BEFORE editing and glUnmapBuffer AFTER, every time.

Exercise Q05 — Matrix Addition, By Hand [Easy]

Objective: Practice matrix addition using two 2x2 matrices, following the lecture's Addition and Subtraction slide.

Task (Question): Add two 2x2 matrices element by element in code, print the resulting matrix to the console, and draw a simple shape to confirm the program still runs correctly.

Hints:

- Matrix addition only works when both matrices have the SAME dimensions.
- You can store a 2x2 matrix as a 4-element float array and add corresponding indices.

Exercise Q06 — glBufferSubData for a Colored Line [Easy]

Objective: Practice the basic glBufferSubData pattern (reserve, then upload twice) on the simplest possible shape.

Task (Question): Draw a GL_LINES segment with two different-colored endpoints, reserving space with ONE glBufferData(NULL) call and uploading position data and color data with TWO separate glBufferSubData() calls.

Hints:

- Reserve space for BOTH blocks in the very first glBufferData call — its `data` argument should be NULL.
- The color block's byte offset must equal the exact size (in bytes) of the position block that came before it.

Exercise Q07 — glArrayElement Basics [Easy]

Objective: Practice the most basic glArrayElement() usage: three hand-typed calls for a triangle.

Task (Question): Draw a triangle using THREE hand-typed glArrayElement() calls inside glBegin(GL_TRIANGLES)/glEnd(), with position data sourced from a VBO.

Hints:

- glArrayElement() must be called between glBegin() and glEnd() — not outside it.
- You still need glEnableClientState(GL_VERTEX_ARRAY) and glVertexPointer() set up beforehand.

Medium Exercises

Exercise Q08 — scale() Applied to Every Vertex of a Shape [Medium]

Objective: Practice writing a scale() matrix function and applying it to ALL vertices of a shape in a loop.

Task (Question): Write a scaleAll() matrix function (following the lecture's Scaling matrix template) and apply it, in a loop, to every vertex of a VBO quad, doubling its size before the first draw.

Hints:

- Loop with a step of 3 (i += 3) since each vertex has 3 floats (x, y, z).
- Double-check that w stays as 1.0f throughout the multiply, matching the lecture's matrix template.

Exercise Q09 — rotateZ() Applied Once [Medium]

Objective: Practice writing a rotateZ() matrix function and applying a single 45-degree rotation to a shape.

Task (Question): Write a rotateZ() matrix function using the lecture's Rz(theta) template, and apply a ONE-TIME 45-degree rotation to every vertex of a VBO quad before its first draw.

Hints:

- Convert degrees to radians before calling cosf()/sinf() — OpenGL's trig functions expect radians.
- The matrix multiply pattern is identical to translate() — only the 16-value matrix template changes.

Exercise Q10 — Combining Translate and Scale [Medium]

Objective: Practice multiplying two matrices together into ONE combined matrix, per the lecture's Combining Transformation slide.

Task (Question): Write a multiply4x4() helper, use it to combine a translation matrix and a scale matrix into ONE matrix, and apply that single combined matrix to every vertex of a triangle.

Hints:

- Write multiply4x4() as a general-purpose function that takes two 16-value arrays and produces a third — you'll reuse this pattern often.
- Apply the TRANSLATION matrix as the first argument and the SCALE matrix as the second, matching the lecture's M_translate . M_scale order.

Exercise Q11 — Color Cube Face via glBufferSubData [Medium]

Objective: Practice combining position and per-vertex color data for a cube face into one VBO.

Task (Question): Build ONE VBO holding a cube face's position data followed by per-vertex color data (via glBufferSubData), drawn with glColorPointer() + glDrawArrays(GL_QUADS, ...).

Hints:

- Give at least two different colors across the four vertices so the smooth blending is visible.
- Reserve the FULL combined size in your very first glBufferData(NULL) call.

Exercise Q12 — Timer-Driven translate() Animation [Medium]

Objective: Practice the full glutTimerFunc + translate() animation pattern from the lecture's modifyPolygon() example.

Task (Question): Animate a triangle so it moves UPWARD once per second, using the translate() matrix function and a self-rescheduling glutTimerFunc callback, matching the lecture's modifyPolygon() structure.

Hints:

- Call glutPostRedisplay() at the end of your timer callback so the movement is actually visible.
- Reschedule the timer as the LAST line inside the callback, exactly like modifyPolygon() does.

Exercise Q13 — Rotation and Homogeneous Coordinates [Medium]

Objective: Practice testing whether a rotation matrix behaves the same way as a translation matrix with respect to w.

Task (Question): Apply the SAME rotateZ() function to both a position (w=1) and a direction (w=0), print both results to the console, and explain in a code comment why rotation affects w=0 vectors when translation does not.

Hints:

- Unlike the translation matrix, the rotation matrix has NO column that gets multiplied only by w — that's why it affects direction vectors too.
- Compare your printed results directly against Example 10's translation-based output to see the difference.

Exercise Q14 — glArrayElement in a Loop, Five Colors [Medium]

Objective: Practice glArrayElement() inside a FOR LOOP on a shape with more vertices than you'd want to type by hand.

Task (Question): Draw a 5-vertex colored pentagon using glArrayElement() inside a for loop (not hand-typed calls), with position and color both uploaded via glBufferSubData.

Hints:

- A GL_POLYGON with 5 vertices closes automatically back to the first vertex — you don't need to repeat it.
- Give each of the 5 vertices a visibly different color so the blending is easy to see.

Hard Exercises

Exercise Q15 — Scaled-Then-Animated Color Cube [Hard]

Objective: Practice combining a one-time scale with continuous per-second translation on a multi-face color object.

Task (Question): Build a three-face color cube (combined position+color VBO via glBufferSubData) that is scaled down by half ONCE at startup, then drifts diagonally once per second using translate().

Hints:

- Apply the one-time scale INSIDE the same block that uploads the buffer for the first time, using your `ready` flag.
- Keep the scale and the translate as two SEPARATE functions/steps rather than combining them into one matrix, to keep the animation logic simple.

Exercise Q16 — Toggleable Face Culling [Hard]

Objective: Practice controlling GL_CULL_FACE interactively to explore winding order and handedness.

Task (Question): Draw two triangles with opposite winding order, and use glutKeyboardFunc so pressing 'f' toggles GL_CULL_FACE on and off, letting the viewer SEE the clockwise triangle appear and disappear.

Hints:

- Keep a global bool tracking whether culling is currently on, and call glutPostRedisplay() after toggling it.
- glCullFace(GL_BACK) culls back-facing (here, clockwise-wound) triangles by default.

Exercise Q17 — Triggering the Error Callback [Hard]

Objective: Practice registering glDebugMessageCallback AND deliberately triggering an error so the callback actually fires.

Task (Question): Register a glDebugMessageCallback exactly as in Example 16, then deliberately call an OpenGL function with an INVALID enum (something guaranteed to be wrong) so the callback fires and prints a diagnostic message.

Hints:

- Any clearly-invalid GLenum value passed to a function like glEnable() will trigger an error.
- Check your terminal/console output, not the graphics window, to see the callback's printed message.

Exercise Q18 — Two Shapes, One Combined Buffer [Hard]

Objective: Practice laying out FOUR separate data blocks (two shapes' worth of position AND color data) inside a single VBO using glBufferSubData.

Task (Question): Combine a pyramid face's position+color data AND a cube face's position+color data into ONE VBO (four glBufferSubData calls total), and draw both shapes with two separate glDrawArrays calls using different vertex/color pointer offsets.

Hints:

- Work out all four byte offsets on paper BEFORE writing the code — position block 1, position block 2, color block 1, color block 2.
- Each glDrawArrays call needs its OWN glVertexPointer/glColorPointer call first, pointing at that shape's specific offsets.

Exercise Q19 — Continuous Y-Axis Rotation With Cleanup [Hard]

Objective: Practice combining continuous rotation, a color VBO, and ESC-key cleanup into one program.

Task (Question): Continuously rotate a colored quad around the Y axis using rotateY(), once per second via glutTimerFunc, with a monitorKeyboard() ESC handler that releases the VBO before exit(0).

Hints:

- rotateY() follows the exact same code shape as rotateZ() — only the matrix template's row/column layout changes.
- Test that pressing ESC actually stops the animation and closes the window cleanly, not just that the rotation looks right.

Exercise Q20 — Capstone: Rotating and Drifting Color Cube [Hard]

Objective: Practice combining TWO different matrix transformations (rotation AND translation) inside the SAME animation callback, on a multi-face color object, with full cleanup.

Task (Question): Build a three-face color cube (combined position+color VBO) that BOTH rotates around Z and drifts diagonally every second (applying rotateZ() then translate() to each vertex inside one timer callback), with ESC-key cleanup releasing the buffer.

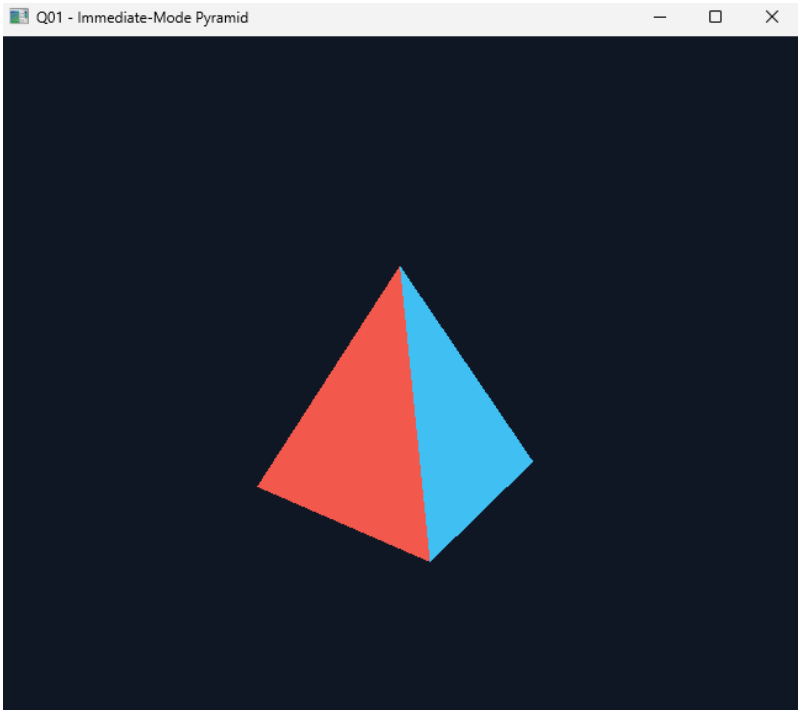
Hints:

- Apply rotateZ() and translate() to the SAME vertex, one after the other, inside your per-vertex loop.
- Count your buffers carefully in the cleanup handler — this capstone uses exactly ONE combined VBO, but make sure you unbind before deleting.

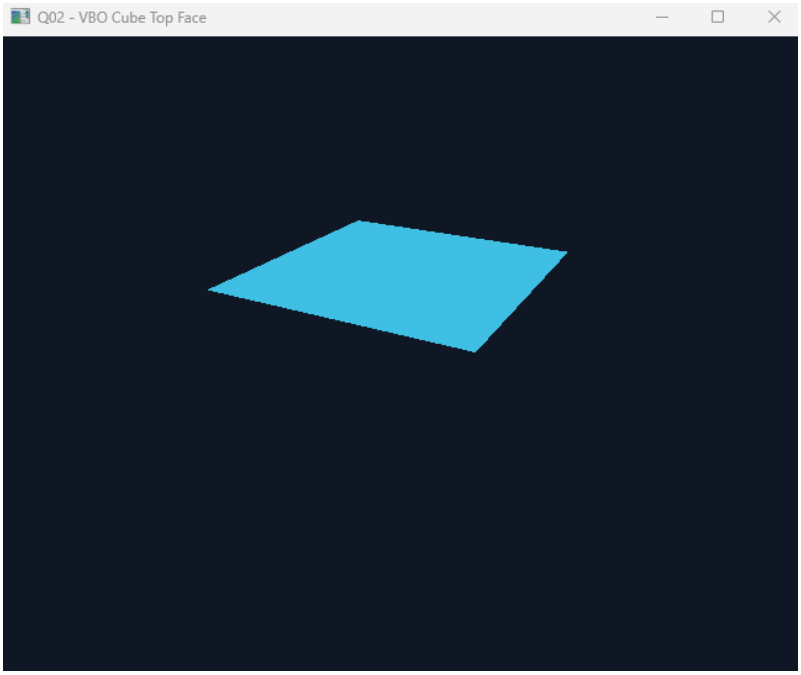
Part C — Outputs / Screen Shots (Ex01–Ex20)

Easy Exercises

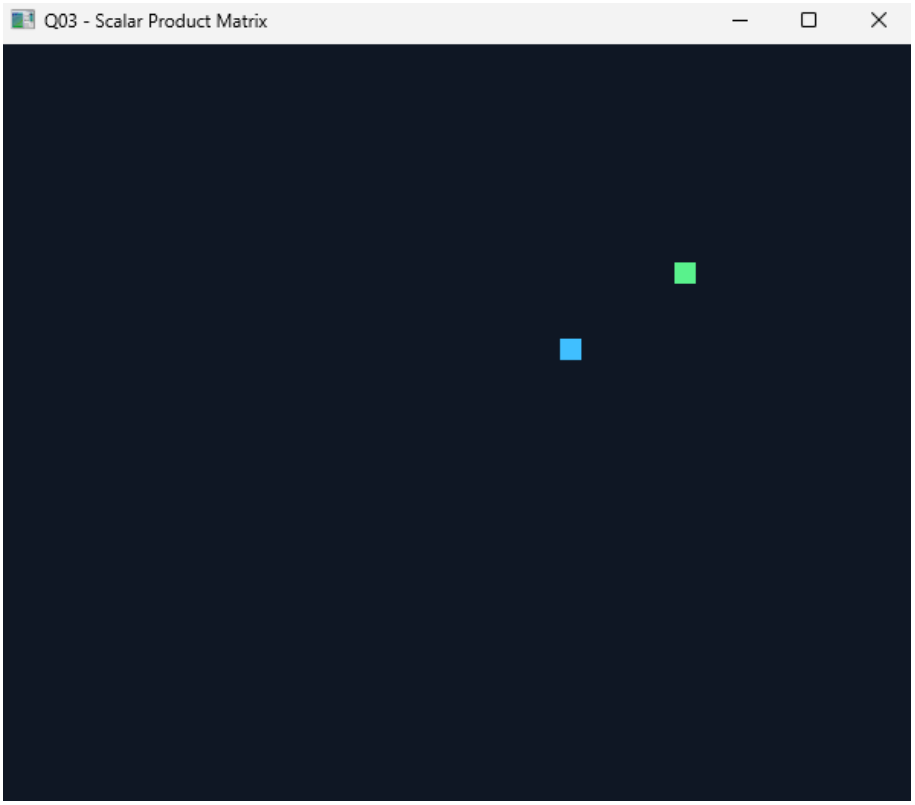
Exercise Q01 — Immediate-Mode 4-Sided Pyramid [Easy]



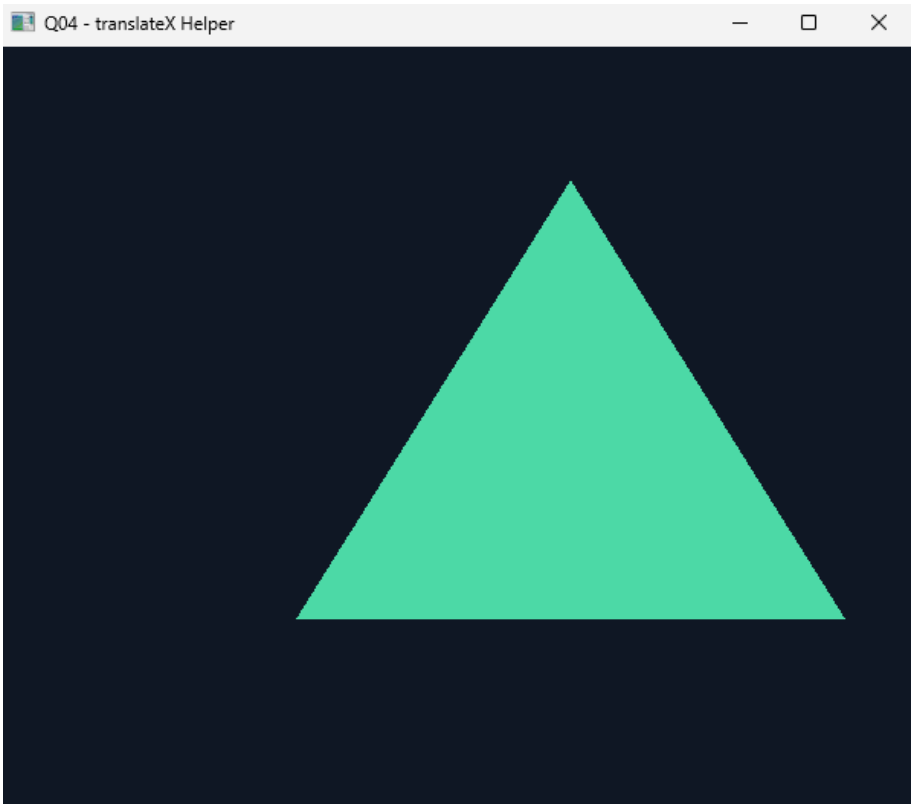
Exercise Q02 — VBO Conversion of One Cube Face [Easy]



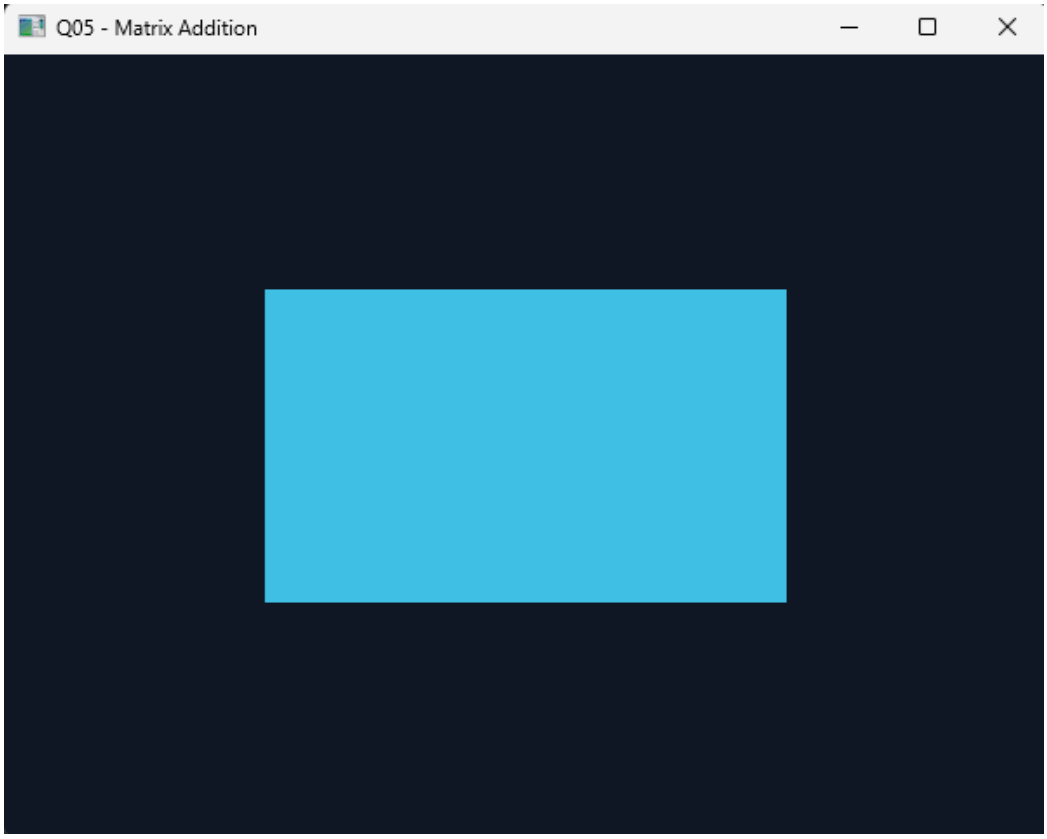
Exercise Q03 — Scalar Product Matrix, By Hand [Easy]



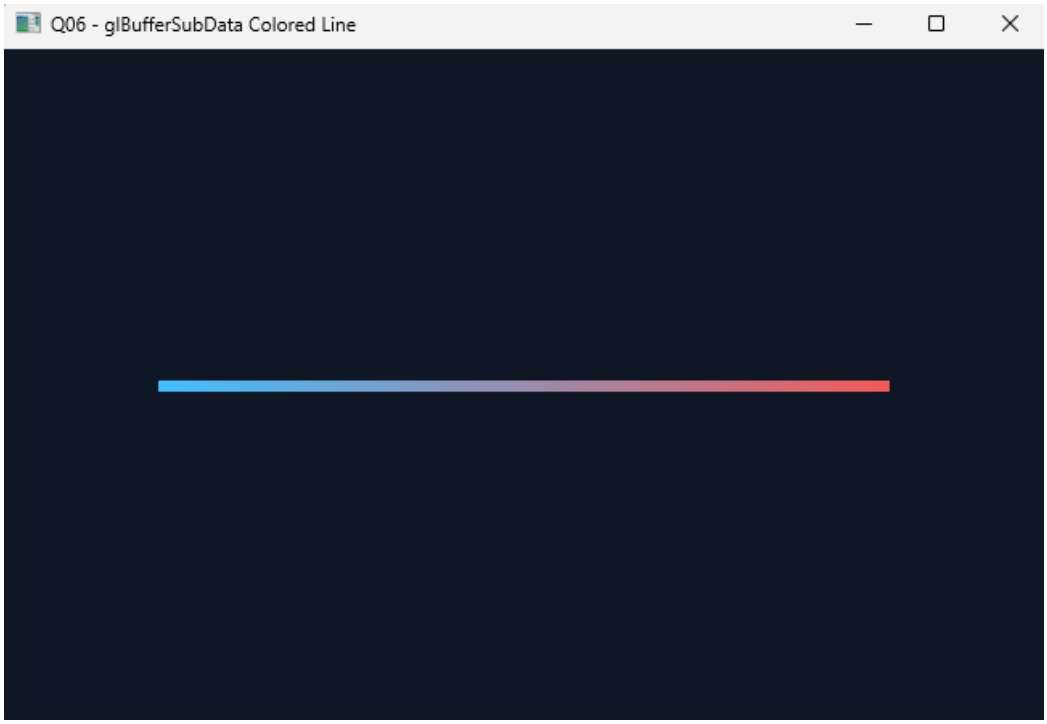
Exercise Q04 — A Simple translateX() Helper [Easy]



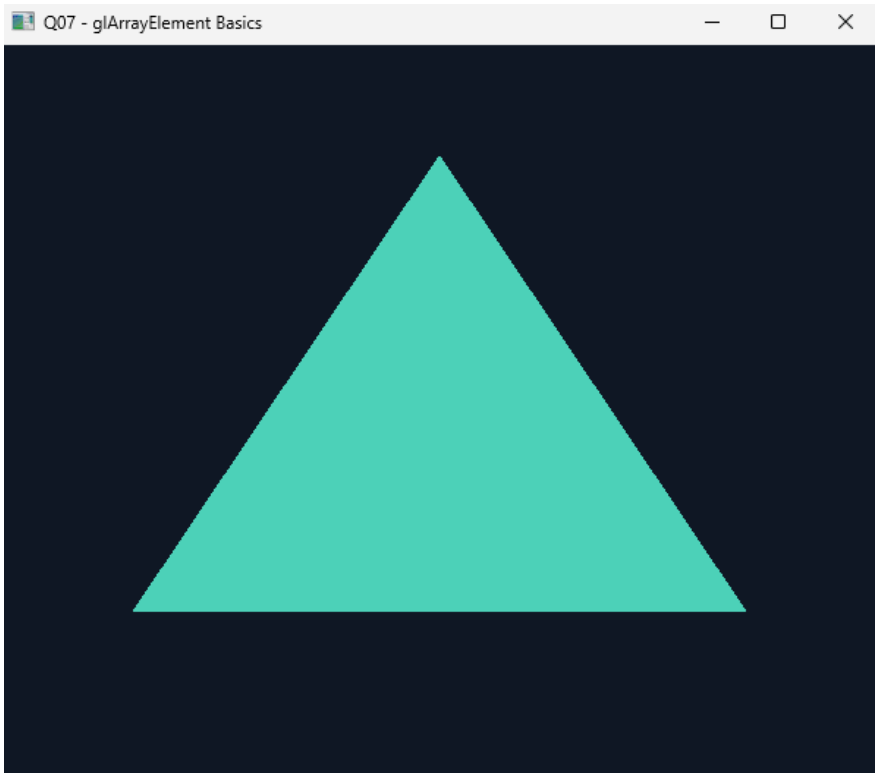
Exercise Q05 — Matrix Addition, By Hand [Easy]



Exercise Q06 — glBufferSubData for a Colored Line [Easy]

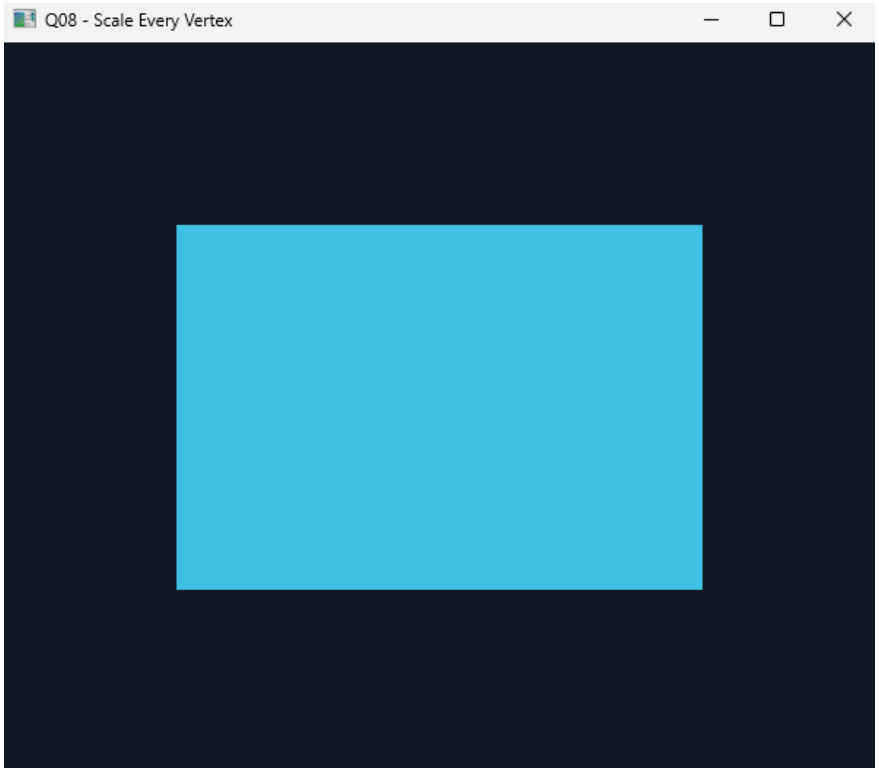


Exercise Q07 — glArrayElement Basics [Easy]

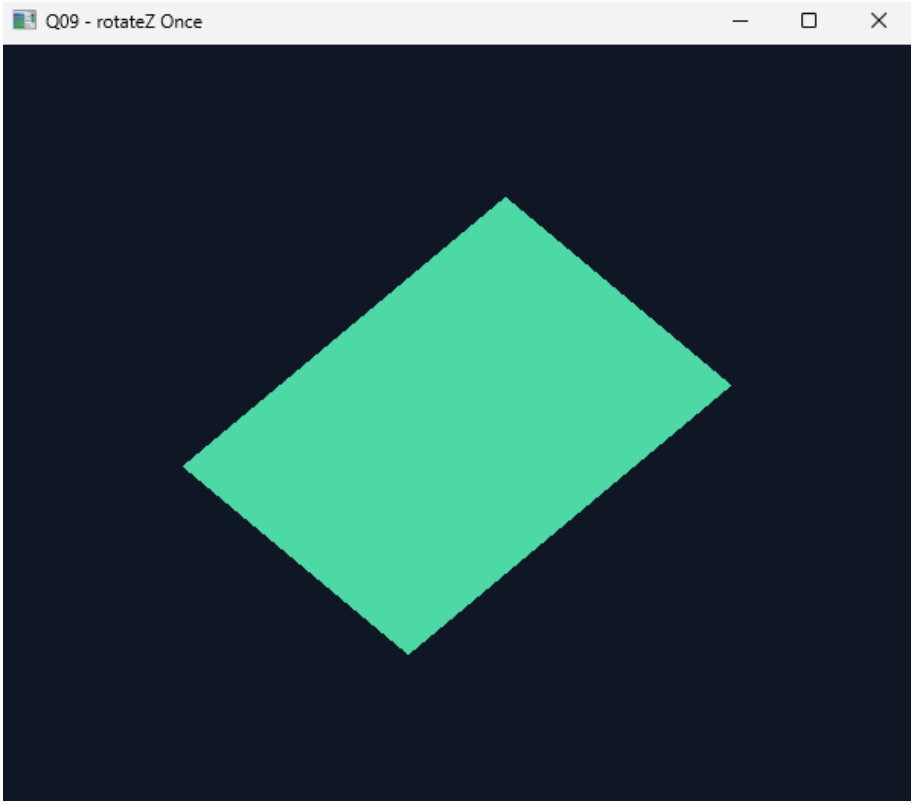


Medium Exercises

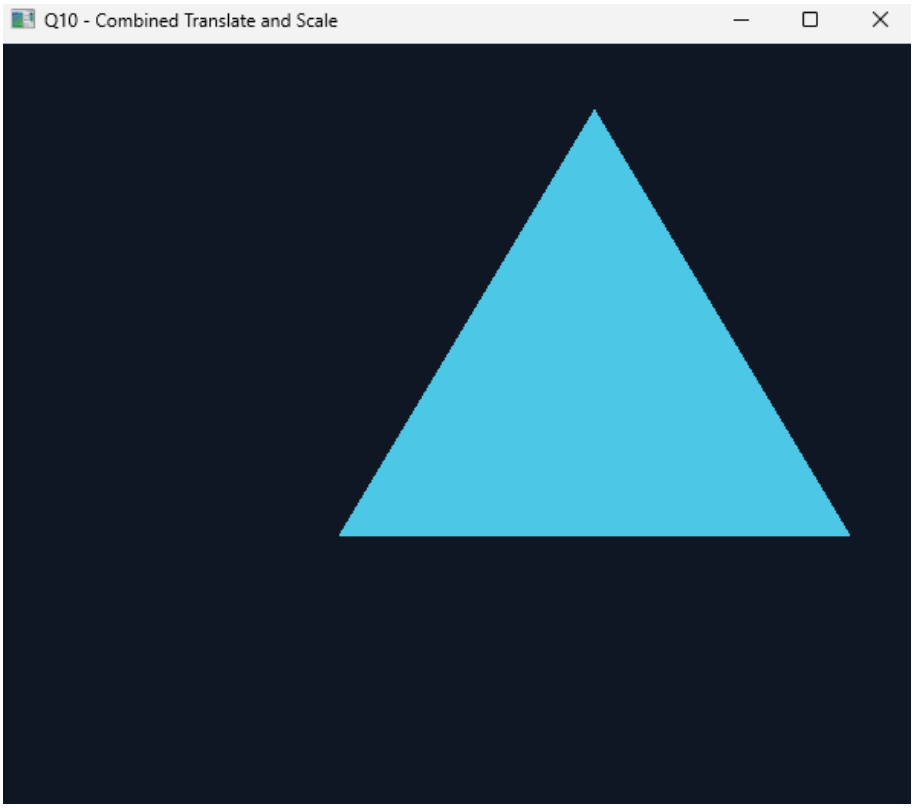
Exercise Q08 — scale() Applied to Every Vertex of a Shape [Medium]



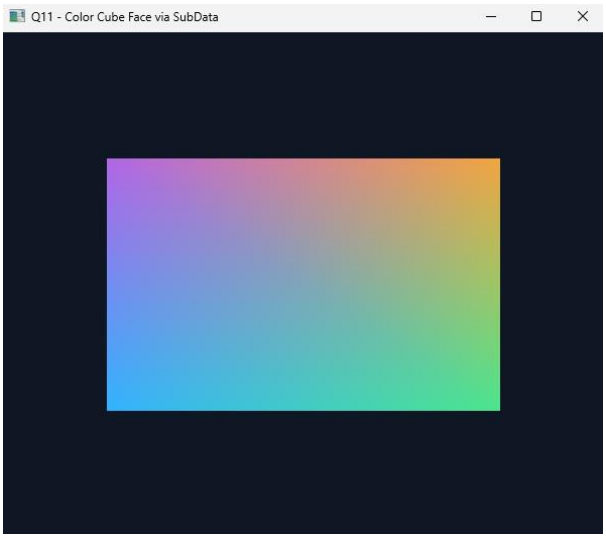
Exercise Q09 — rotateZ() Applied Once [Medium]



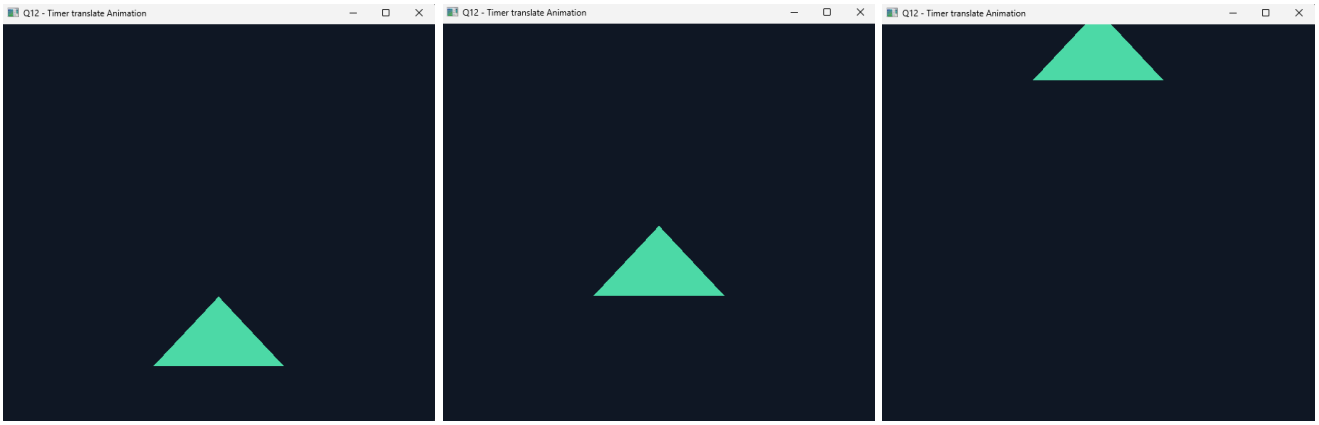
Exercise Q10 — Combining Translate and Scale [Medium]



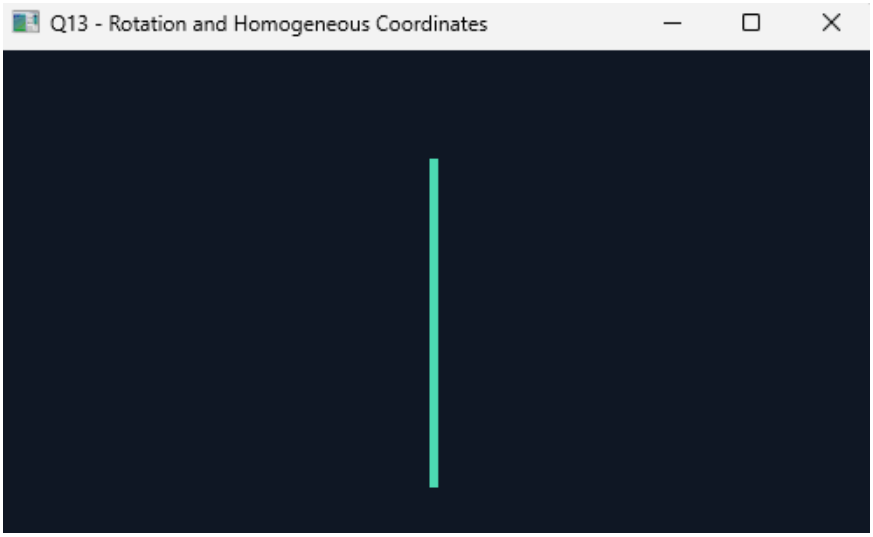
Exercise Q11 — Color Cube Face via glBufferSubData [Medium]



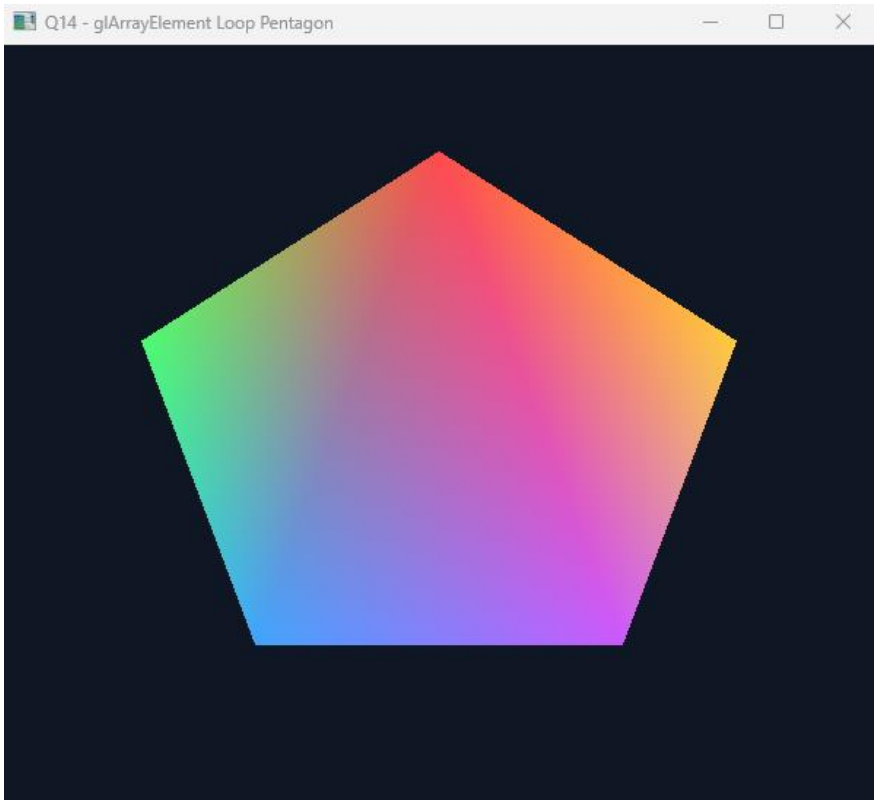
Exercise Q12 — Timer-Driven translate() Animation [Medium]



Exercise Q13 — Rotation and Homogeneous Coordinates [Medium]

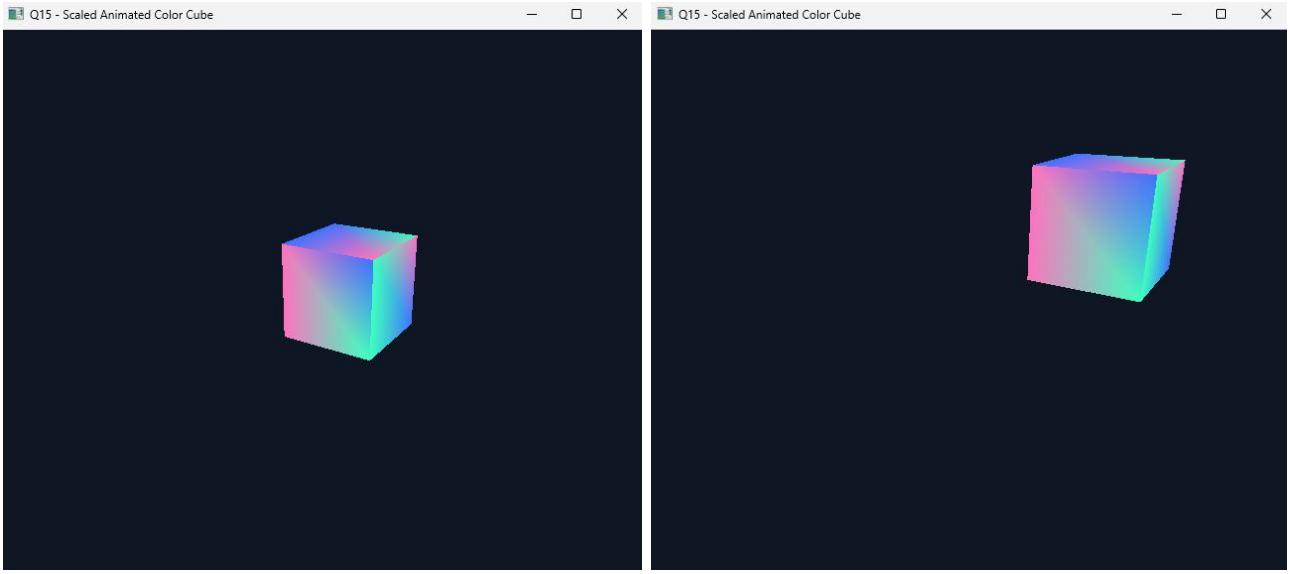


Exercise Q14 — glArrayElement in a Loop, Five Colors [Medium]

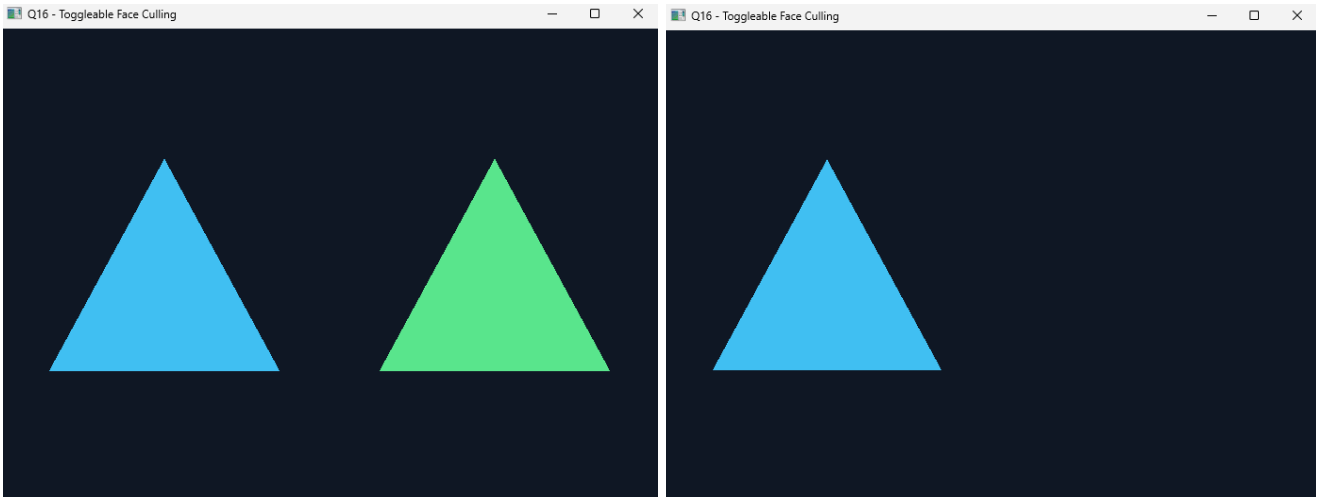


Hard Exercises

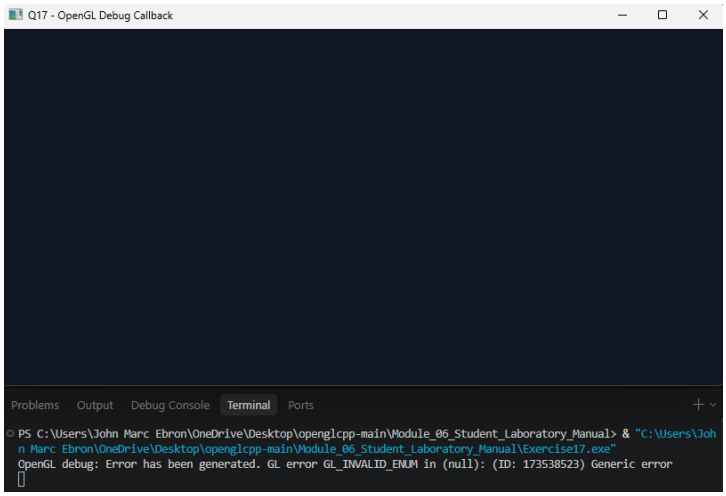
Exercise Q15 — Scaled-Then-Animated Color Cube [Hard]



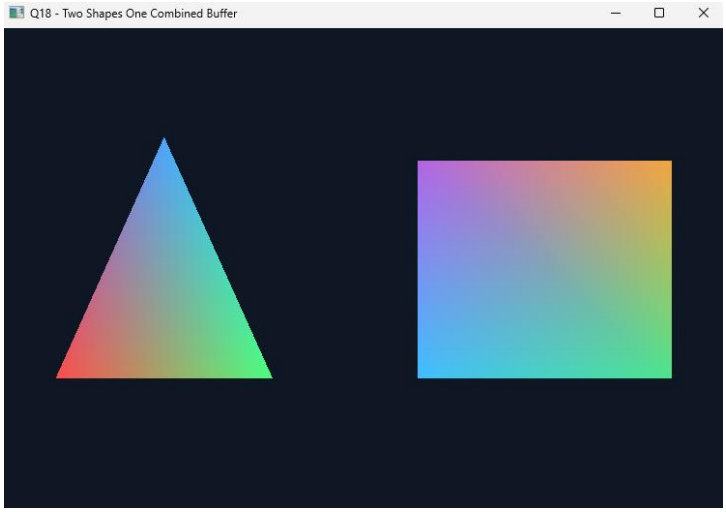
Exercise Q16 — Toggleable Face Culling [Hard]



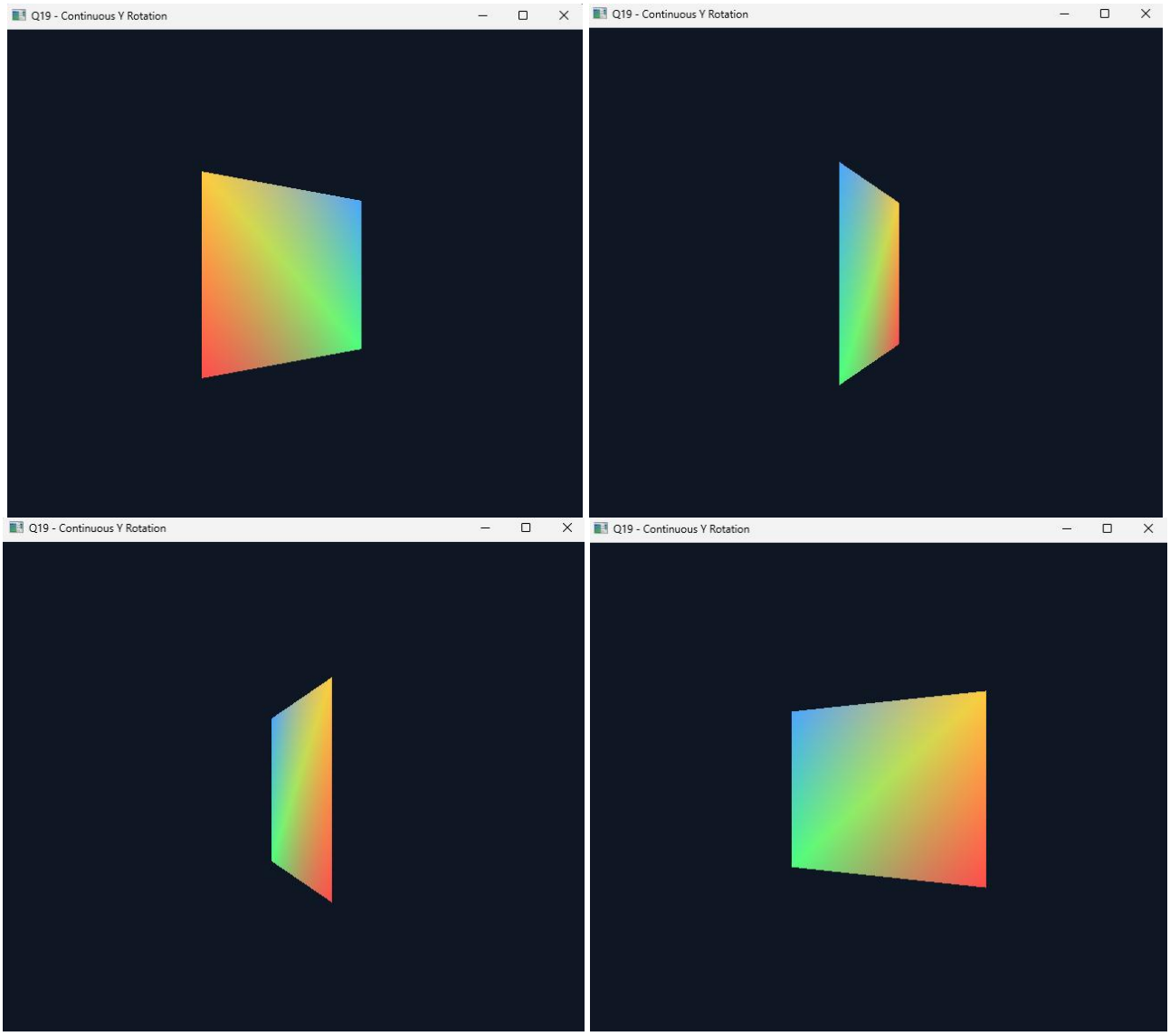
Exercise Q17 — Triggering the Error Callback [Hard]



Exercise Q18 — Two Shapes, One Combined Buffer [Hard]



Exercise Q19 — Continuous Y-Axis Rotation With Cleanup [Hard]



Exercise Q20 — Capstone: Rotating and Drifting Color Cube [Hard]

